

YubiKey + GnuPG

Setup Guide for Gentoo

*Install tools, create an OpenPGP key, move it onto a YubiKey,
and encrypt secrets for rclone / AWS scripts.*

Compiled from a working setup session on 27 September 2026.

Recipient used in examples: Cv25519 encrypt subkey 6BDAEBAB20864DEF (Alan <ab@ab.com>).

1. What this setup is

A YubiKey can hold an OpenPGP private key. GnuPG talks to the stick through `scdaemon` / `gpg-agent`. The private material never leaves the card. Decrypting a file requires the physical key, the OpenPGP User PIN, and (if you enable it) a touch on the gold disc.

This is **asymmetric** encryption (`gpg --encrypt -r KEYID`). It is not the same as the earlier `gpg --symmetric` passphrase blob. A YubiKey does not unlock a symmetric message. Re-encrypt old secrets to the card key.

Goal	Mechanism
Encrypt a secret	<code>gpg --encrypt --armor -r 6BDAEBAB20864DEF</code>
Decrypt	YubiKey present + User PIN + optional touch
Keep MFA (GitHub, Google, etc.)	Do not touch FIDO / OTP / OATH applets
Script secrets	Public-key ciphertext in a file or base64 blob

2. YubiKey PINs and MFA — what you can change safely

PINs are per applet. Changing OpenPGP settings does not rewrite FIDO passkeys.

Application	What it unlocks	PINs
OpenPGP (GPG)	Sign / encrypt / auth keys on the card	User PIN, Admin PIN, optional Reset Code
PIV	PIV slots (smart card, some age plugins)	PIN, PUK
FIDO2 / WebAuthn	Browser and OS passkeys	FIDO PIN
Yubico OTP / OATH	OTP paste and TOTP in Authenticator	Not the OpenPGP PIN

Factory defaults to change immediately

OpenPGP User PIN	123456
OpenPGP Admin PIN	12345678
PIV PIN	123456
PIV PUK	12345678

There is one OpenPGP User PIN for sign, encrypt, and authenticate. You cannot give decrypt its own PIN. You can require a physical touch per slot instead.

Operations that will break existing MFA

Command / action	Effect
<code>ykman fido reset</code>	Wipes all passkeys
Disable FIDO USB or NFC	Sites cannot see the key
<code>ykman oath reset</code>	Wipes Authenticator TOTP accounts
Overwrite OTP slot 1 or 2	Breaks Yubico OTP 2FA

Command / action	Effect
Full device reset	Wipes every applet
gpg --card-edit factory-reset / ykman openpgp reset	Wipes GPG keys only — FIDO MFA stays

Safe on a key that already does web MFA: generate GPG keys, keytocard, change OpenPGP PINs, set OpenPGP touch policy, enable the OpenPGP USB app. Do not run fido reset, oath reset, or ykman config usb --disable FIDO.

3. Gentoo packages

Official wiki: wiki.gentoo.org/wiki/YubiKey and wiki.gentoo.org/wiki/YubiKey/GPG.

```
emerge --ask \
  sys-apps/pcsc-lite \
  app-crypt/ccid \
  app-crypt/yubikey-manager \
  app-crypt/gnupg

# GUI Authenticator (replaces retired yubikey-manager-qt)
emerge --ask app-crypt/yubioath-flutter-bin

# optional
emerge --ask sys-auth/yubico-piv-tool
```

USE flags

Put this in /etc/portage/package.use:

```
app-crypt/gnupg smartcard usb
```

- smartcard — builds sdaemon (required for GPG + YubiKey).
- usb — sdaemon can talk CCID directly.
- yubikey-manager is the ykman CLI and pulls pcsc-lite + ccid.

pcscd service

```
# OpenRC
rc-update add pcscd default
rc-service pcscd start

# systemd
systemctl enable --now pcscd
```

Add your user to plugdev / usb / pcscd if udev does not grant access. Re-plug the key after emerging ccid.

GPG vs pcscd (the usual Gentoo footgun)

ykman wants pcscd. sdaemon wants exclusive CCID. They fight unless you share the reader. Put this in ~/.gnupg/scdaemon.conf:

```
disable-ccid
pcsc-shared

gpgconf --kill all
gpg --card-status
ykman info
```

4. Official downloads (any distro)

- Hub: yubico.com/support/download/
- Linux notes: support.yubico.com — Installing Yubico Software on Linux
- Authenticator: [Flathub com.yubico.yubioath](https://flathub.com/yubico.yubioath), or Yubico’s Linux tarball
- ykman releases: github.com/Yubico/yubikey-manager/releases

YubiKey Manager GUI is retired. Use Authenticator + ykman. pcscd must be running for OATH, PIV, and GPG-over-PC/SC.

5. Create the OpenPGP key and decide where it lives

Piece	Where it lives	Purpose
Certify (master) private key	Offline backup only (LUKS USB / air-gap)	Add UIDs, revoke, issue subkeys
Sign / Encrypt / Auth private subkeys	YubiKey OpenPGP slots	Daily decrypt, sign, SSH
Public key	This machine, key servers, git	<code>gpg --encrypt -r ...</code>
Stubs on disk (ssb>)	<code>~/.gnupg</code>	GPG knows “use the card”

Do not generate the only copy on the YubiKey unless you accept that a lost stick is a lost identity. Generate on disk, back up the master, move subkeys with `keytocard`, then delete on-disk private subkeys.

Generate

```
gpg --full-generate-key
# or gpg --expert --full-generate-key

# Sensible YubiKey 5 layout:
#   Primary: Certify only (Ed25519 or RSA 4096)
#   Subkeys: Sign (Ed25519), Encrypt (Cv25519), Auth (Ed25519)
#   Expiration: 1-2 years (extend later with the master)

gpg --list-secret-keys --keyid-format long
```

The passphrase you type here protects the **on-disk master**. It is not the YubiKey User PIN. After the secrets live only on the card, decrypt asks for the PIN, not this passphrase.

Backup before moving anything

```
gpg --export-secret-keys --armor KEYID > master-private.asc
gpg --export --armor KEYID > public.asc
gpg --export-secret-subkeys --armor KEYID > subkeys-private.asc # optional
chmod 600 master-private.asc public.asc
# Keep master-private.asc offline. Also keep ~/.gnupg/openpgp-revocs.d/ offline.
```

Change default card PINs first

```
gpg --card-status
gpg --card-edit
# admin → passwd → change User PIN (123456) and Admin PIN (12345678)

# or
ykman openpgp access change-pin
ykman openpgp access change-admin-pin
```

Move subkeys onto the YubiKey

```
gpg --edit-key KEYID
# inside:
# list
# key N          (star the [E] / cv25519 subkey)
# keytocard      (slot 2 = Encryption)
# key            (deselect)
# repeat for [S] slot 1 and [A] slot 3
# save
```

Strip private material from this machine

```
gpg --delete-secret-keys KEYID # after the offline backup exists
gpg --import public.asc
gpg --card-status # recreates stubs
gpg --list-secret-keys --keyid-format long
```

Marker	Meaning
sec / ssb	Private key still on this machine — GPG will use the key passphrase
sec#	Master private key is not on this machine
ssb>	Subkey is a card stub — decrypt uses the YubiKey

If decrypt still asks for the *key creation password*, the encrypt subkey is still on disk. The line you want for encrypt is `ssb>` on the Cv25519 [E] key. Leftover files in `~/.gnupg/private-keys-v1.d/` will keep unlocking with the old passphrase.

Second computer

Copy `public.asc` (or `gpg --export`), import it, plug the same YubiKey, run `gpg --card-status`. No private key file is required on the second machine.

6. Which key ID to encrypt to

Encrypt to the identity (email or primary fingerprint). GPG selects the encrypt-capable subkey. You may also pass the [E] subkey ID explicitly. Do not use [S] or [A] IDs.

```
gpg --list-keys --keyid-format long

# Example layout:
# pub  ed25519/AAAAAAAA11111111 [C]
# uid  Alan <ab@ab.com>
# ssb  ed25519/BBBBBBBB22222222 [S]
# ssb  cv25519/6BDAEBAB20864DEF [E] ← this one
# ssb  ed25519/CCCCCCC33333333 [A]

gpg --encrypt --armor -r 'Alan <ab@ab.com>'
gpg --encrypt --armor -r 6BDAEBAB20864DEF
```

Confirm the ciphertext without touching the card

`gpg --list-packets` tries to unwrap the session key if a secret is available. On a pipe that steals stdin and can lock the PIN. Cancel pinentry when you only want to inspect:

```
printf 'test\n' | gpg --encrypt --armor -r 6BDAEBAB20864DEF \
| gpg --list-packets --pinentry-mode cancel
```

Success looks like: `pubkey enc packet`, algo 18, keyid 6BDAEBAB20864DEF. If you see `symkey enc packet`, the file is still `--symmetric`.

7. Decrypt the right way

Pinentry **mode** is not the same as the pinentry **program**. Mode `ask` launches a real PIN dialog. Mode `loopback` makes `gpg` read a passphrase from the caller — and if ciphertext is on stdin, that “passphrase” is garbage. The LED can still blink. Then you get Bad PIN / PIN blocked.

Working pattern

```
export GPG_TTY=$(tty)
gpgconf --launch gpg-agent

printf 'test\n' | gpg --encrypt --armor -r 6BDAEBAB20864DEF > /tmp/t.asc
gpg --decrypt --pinentry-mode ask /tmp/t.asc
rm /tmp/t.asc
```

Expected order: PIN dialog → LED → press the gold disc (if touch is on) → plaintext. A pipe into `gpg --decrypt --pinentry-mode ask` can work if pinentry is a GUI. Curses pinentry on the same TTY as the pipe can still steal stdin. Prefer a file.

Curses pinentry on Gentoo

```
# ~/.gnupg/gpg-agent.conf
pinentry-program /usr/bin/pinentry-curses

gpgconf --kill gpg-agent
export GPG_TTY=$(tty)
ls /usr/bin/pinentry*
```

Remove `pinentry-mode loopback` from `~/.gnupg/gpg.conf` if it was added for the old symmetric snippets. Loopback is for passphrase blobs, not the card.

8. Require a button press

Touch is off by default on the encrypt slot. Turn it on with `ykman` (Admin PIN). `pcscd` must be running.

```
gpgconf --kill all
rc-service pcscd start      # or: systemctl start pcscd
ykman openpgp keys set-touch enc on
```

Policy	Behavior
off	PIN only
on	PIN + touch every decrypt
cached	Touch once, skipped for a few seconds
fixed / cached-fixed	Same, cannot lower without an OpenPGP reset

```
ykman openpgp keys set-touch sig on
ykman openpgp keys set-touch aut cached
```

9. PIN blocked

Three failed User PIN attempts lock OpenPGP. Piped loopback and list-packets without cancel are how this usually happens. Stop decrypting until you unblock.

```
gpg --card-status | grep -i pin
ykman openpgp info

gpg --card-edit
# admin → passwd → 2 unblock PIN
# enter Admin PIN, set a new User PIN

# or
ykman openpgp access unblock-pin
```

If the Admin PIN is also locked, only an OpenPGP reset remains. That wipes GPG keys on the card, not FIDO MFA. You must keytocard again from the offline master backup.

```
ykman openpgp reset
```

10. Encrypt / decrypt scripts

Payload format: space-separated `KEY=VALUE` tokens, any number of them. Values must not contain spaces. Source the decrypt script so names exist in the current shell. Assignments are not exported; child processes (`rcclone`) will not see them unless you export or pass flags explicitly.

pwencrypt.sh

```
#!/usr/bin/env bash
# Encrypt space-separated KEY=VALUE secrets to the YubiKey key.
# Prints one base64 line.
#
#   ./pwencrypt.sh ACCESS_KEY_ID=AKI... SECRET_ACCESS_KEY=wJal... OTHER=foo
#   echo 'ACCESS_KEY_ID=AKI... SECRET_ACCESS_KEY=wJal...' | ./pwencrypt.sh

recipient="${GPG_RECIPIENT:-6BDAEBAB20864DEF}"

if [[ $# -gt 0 ]]; then
    pairs=("$@")
else
    read -r -a pairs <<< "$(tr '\n\t' ' ')"
fi

plain=""
for pair in "${pairs[@]"; do
    [[ -z $pair ]] && continue
    [[ $pair == *=* ]] || { echo "expected KEY=VALUE: $pair" >&2; exit 1; }
    key=${pair%%=*}
    [[ $key =~ ^[A-Za-z_][A-Za-z0-9_]*$ ]] || { echo "bad name: $key" >&2; exit 1; }
    plain+="${plain:+ }$pair"
done

[[ -n $plain ]] || { echo "no input" >&2; exit 1; }

printf '%s' "$plain" \
    | gpg --encrypt --armor --quiet --no-greeting -r "$recipient" \
    | base64 --wrap 0
echo
```

pwdecrypt.sh

Replace the heredoc with the line printed by pwencrypt.sh, folded if you like. `tr -d '[:space:]'` is safe on the **outer base64** only — never on raw PGP armor.

```
#!/usr/bin/env bash
#####
## Source this file: source ./pwdecrypt.sh
## Sets KEY=VALUE from a space-separated encrypted blob.
#####
blob=$(tr -d '[:space:]' <<'EOF'
PASTE_BASE64_FROM_pwencrypt_HERE
EOF
)

[[ -n ${GPG_TTY:-} ]] || export GPG_TTY=$(tty)

_tmp=$(mktemp)
trap 'rm -f "$_tmp"' RETURN
printf '%s' "$blob" | base64 -d > "$_tmp"

plain=$(gpg --decrypt --pinentry-mode ask --quiet --no-greeting "$_tmp") || return 1

read -r -a pairs <<< "$plain"
for pair in "${pairs[@]"; do
    [[ -z $pair ]] && continue
    key=${pair%*=*}
    val=${pair#*=*}
    printf -v "$key" '%s' "$val"
done
```

Embedding a long base64 blob

The encrypt pipeline is: GPG armor (must keep its newlines) then base64 --wrap 0 (newlines optional). Strip whitespace only from the outer base64. Do not wrap encrypt in echo \$(...); that collapses armor.

```
blob=$(tr -d '[:space:]' <<'EOF'
LS0tLS1CRUdJT1BQR1AgTUVU00FHRS0tLS0t...
EOF
)
```

If the stored string starts with LS0t it is outer base64 of -----BEGIN. If it starts with -----BEGIN PGP, it is raw armor: do not strip spaces and do not base64 -d.

11. AWS rclone launcher

Keep secrets in ~/.config/rclone-aws-creds.gpg as two lines (ACCESS_KEY_ID=... and SECRET_ACCESS_KEY=... or two raw values). Encrypt that file to the card key, not --symmetric.

```
# Convert an old symmetric file:
gpg --decrypt ~/.config/rclone-aws-creds.gpg \
| gpg --encrypt --armor -r 6BDAEBAB20864DEF \
-o ~/.config/rclone-aws-creds.gpg.new
mv ~/.config/rclone-aws-creds.gpg.new ~/.config/rclone-aws-creds.gpg

# Or from plaintext:
printf '%s\n' \
'ACCESS_KEY_ID=AKIA...' \
'SECRET_ACCESS_KEY=...' \
| gpg --encrypt --armor -r 6BDAEBAB20864DEF \
-o ~/.config/rclone-aws-creds.gpg
```

Decrypt section used by the launcher (before detach, so pinentry still has a TTY):

```
export GPG_TTY="${GPG_TTY:-$(tty)}"
echo "Decrypting credentials (YubiKey PIN, then touch if ENC touch is on)..."
creds=$(gpg --pinentry-mode ask --quiet --no-greeting --decrypt "$CREDENTIALS_FILE")
if [[ $? -ne 0 || -z "$creds" ]]; then
    echo "Error: Failed to decrypt $CREDENTIALS_FILE" >&2
    exit 1
fi
```

Parse KEY=value or two plain lines, unset the raw blob, pass keys only as rclone flags, hide the secret in the printed command, unset after exit. Decrypt happens before detach.

12. Troubleshooting

Symptom	Cause / fix
Decrypt uses key-creation password, not PIN	Encrypt subkey still on disk (ssb not ssb>). keytocard, delete-secret-keys, import public, card-status.
YubiKey lights, then Bad PIN, no prompt	loopback + ciphertext on stdin. Use --pinentry-mode ask and a file. Remove loopback from gpg.conf.
PIN blocked	Unblock with Admin PIN. Do not keep retrying decrypt.
invalid armor header: ...END PGP MESSAGE	Newlines stripped from inner GPG armor (echo \$(gpg --armor) or tr -d on armor). Only strip outer base64.
gpg --card-status: no device	pcscd down, OpenPGP USB app disabled, or CCID fight. Enable OpenPGP, start pcscd, scdaemon.conf disable-ccid + pcsc-shared.
ykman works, gpg does not (or the reverse)	Same CCID fight. gpgconf --kill all after editing scdaemon.conf.
list-packets reports Bad PIN	It tried to unwrap the session key. Add --pinentry-mode cancel.
No touch required	ykman openpgp keys set-touch enc on

13. What not to put in the script

- GPG master private key, YubiKey Admin PIN, or FIDO PIN.
- export of long-lived secrets in an interactive parent shell if you can avoid it.
- --passphrase on the command line (visible in ps).
- HashiCorp Vault, for this job. A few static AWS/rclone secrets on one Gentoo box do not justify running and unsealing a server.

Closer upgrades than Vault if the homemade blob becomes annoying: SOPS+age, dotenvx, or pass/passage with the same YubiKey. Env vars remain a last hop into a short-lived child, not a place to store secrets.

14. Quick reference

```
ykman info
ykman openpgp info
gpg --card-status
gpg --list-secret-keys --keyid-format long

# encrypt
printf '%s' 'ACCESS_KEY_ID=AKI... SECRET_ACCESS_KEY=wJal...' \
| gpg --encrypt --armor -r 6BDAEBAB20864DEF | base64 --wrap 0

# decrypt file
export GPG_TTY=$(tty)
gpg --decrypt --pinentry-mode ask file.gpg

# source blob helper
source ./pwdecrypt.sh
```

This document is a personal runbook from one working setup. Key IDs, emails, and paths are examples from that session. Rotate anything that has been pasted into chat logs.